Field Hours

CSCE 431 Software Engineering

Original notes for this site. Same topics as the catalog course. Not a reprint of another university's lectures, and not academic credit.

Lecture 1 · 50 minutes

A requirement a test can fail

A requirement says what the system must do in a sentence a stranger can check. 'Fast' is not a requirement. 'The report for one clinic day finishes in under three seconds on the stated machine' can fail a test. The test is the requirement made executable.

Write the failure before the code. If you cannot say what the wrong output looks like, you do not yet know what you are building. CSCE 606 will argue about process. This course is the sentence and the test.

Two ways to ask for the same feature.

Not yet a requirement

The search should feel smart

Users will love it

A requirement

A misspelled last name still finds the patient

The test plants the typo

One clinic rule

1. The desk asks: find a patient by last name, and a single wrong letter should still show that patient in the first ten rows.

2. The test inserts 'Nguyen' and queries 'Nguyan'. The assertion is that the row appears in the top ten.

3. A search that only does exact match fails this test today. That failure is the start of the work, not a surprise at the demo.

Problem 1. Rewrite 'the app must be secure' into something a test can fail.

Answer. Pick one threat. Example: a request with another patient's id returns no row for a user who is not on that patient's care team. The test sends that request and expects a refusal. 'Secure' names no request and no refusal.

Problem 2. Why does CSCE 606 list this course?

Answer. 606 assumes you can already write a checkable requirement and a test. It spends its weeks on process, review, and change. It should not be the first time you replace an adjective with a number.

Lecture 2 · 50 minutes

A build you can repeat

A build turns the source into the thing you run, with one command, on a machine that is not yours. If the steps live in someone's memory, the project does not build. Pin the tools. Record the command. A new checkout that fails is a bug in the project, not in the newcomer.

Version control stores the history of that source. A commit is a snapshot with a reason. The main line should build. A branch is where it is allowed not to, and it comes back only when the tests you wrote in the previous lecture pass.

From a clean checkout.

Clone -> One setup command -> One test command -> The same result on a second machine

A laptop that is not the author's

1. The author has a library installed globally. The checkout on a lab machine does not.

2. The setup command installs the declared dependencies. The test command does not rely on the global library.

3. If the lab machine still fails, the declaration is incomplete. Add the missing piece to the project, not to a private note.

Problem 1. The tests pass on your laptop and fail in the clean checkout. Where is the defect?

Answer. In the project: an undeclared dependency, a local file that was never committed, or a path that only exists on your machine. 'Works for me' is the report of that defect.

Problem 2. Why is a commit message that says 'fix' not enough?

Answer. The history is how the next person, often you, learns why the line changed. Name the requirement that failed. The diff shows what. The message should say why.

Lecture 3 · 50 minutes

Tests at the boundary

A unit test calls one piece with an input you chose and checks the output. It does not start the whole system. An integration test crosses a boundary: the database, the file, the other module. You need both. A pile of unit tests that never touch the boundary will pass while the boundary is wrong.

A regression test is a unit or integration test that used to fail. Keep it. Deleting it because the bug 'cannot happen again' is how it happens again. The test is the memory.

What each test is allowed to touch.

Unit: one function, inputs you invent

Integration: a real boundary, a fixture you control

A manual click is neither, until you write it down

The typo search, two levels

1. Unit: the distance function returns 1 for Nguyen against Nguyen with one letter changed. No database.

2. Integration: the query against a fixture table returns that patient in the top ten.

3. A click in the browser is a third check. It is not a substitute for the first two, because a click is slow and easy to skip.

Problem 1. All unit tests pass and the clinic still gets no rows. What kind of test was missing?

Answer. A test across the boundary that actually failed: the query, the connection, or the column name. The unit tests never saw it. Add the integration test that would have gone red.

Problem 2. You fixed a bug and deleted the test that caught it, to keep the suite small. What did you lose?

Answer. The proof that the bug stays fixed. The next edit can reintroduce it silently. Keep the test. Shrink the production code if the suite is slow, not the assertion.

Lecture 4 · 50 minutes

A module boundary

A module hides a decision. The rest of the program depends on the behavior you promised, not on the data layout inside. If every file reaches into every other file's tables, a change to a column is a change to the program.

The boundary is a small set of functions. Inputs, outputs, and the errors. What it does not promise is part of the design. CSCE 606 will talk about responsibility. Here, draw the box before you add a second feature.

Who knows the table shape.

No boundary

The page reads the columns itself

A renamed column breaks the page

A boundary

The page asks for a patient

The table can change behind the function

Finding a patient

1. The page calls find_patients(last_name). It does not know the column is called surname.

2. The module runs the query and returns records with the fields the page was promised.

3. Renaming the column edits the module and its tests. The page stays. If the page imported the column name, the rename is a hunt.

Problem 1. A function both draws the button and updates the database. What is the practical failure?

Answer. You cannot test the update without the drawing, and you cannot change the drawing without risking the update. Split them. The drawing calls the update. Each has its own tests.

Problem 2. Is a smaller file always a better module?

Answer. No. A module is a hidden decision, not a line count. Ten tiny files that all know the same table are one module pretending to be ten.

Lecture 5 · 50 minutes

Review

A review is another person reading the change against the requirement, before it lands on the main line. The author does not review alone. The questions are concrete: which test covers this, what happens on the empty input, what did you refuse to change.

Comments on taste are optional. Comments on a missing test are not. If the review always says 'looks good' in under a minute, it is a signature, not a review. CSCE 606 will measure this. Practice it once on a small change.

A change that lands.

1. The requirement and the failing test exist

2. The patch makes the test pass

3. Someone else reads both

4. Then it merges

The typo patch

1. The reviewer asks for the test that plants Nguyen and queries Nguyen with one letter wrong.

2. The author has the test. The reviewer runs it, or reads it and the assertion.

3. The reviewer also asks what a one-letter first name does. If the author has no answer, that is a second requirement, not a reason to block forever, but it is written down.

Problem 1. The author says the change is too small to review. When is that wrong?

Answer. When the change can break a requirement, which is almost every change that touches behavior. A typo in a comment can skip the review. A change to a query cannot. The size in lines is the wrong measure.

Problem 2. What should a review comment cite?

Answer. A requirement, a test, or a failure mode. 'I would have written it differently' is not a defect unless you can say what the different version does that this one does not.

Lecture 6 · 50 minutes

Change

A change request arrives after the first version works. You do not silently absorb it into the same requirement.

Write the new sentence, say what it costs, and decide. A log of decisions is the difference between a project and a folder of files.

CSCE 606 assumes this habit and adds maintenance, release, and the arguments teams actually have. Come in able to point at one requirement, one test, and one review.

A decision, written.

The new sentence

What it breaks or costs

Accepted or refused, and why

Add date of birth to the search

1. The new sentence: a query may include a date of birth, and a wrong date must not hide a correct name match. It ranks lower. It does not disappear.

2. Cost: the index and the unit test grow. The old name-only test must still pass.

3. You accept it and add the test before the code. If you refuse it, the refusal is a sentence in the log, not a forgotten chat message.

Problem 1. A stakeholder asks for a change in the hallway and you implement it the same hour. What is missing?

Answer. The sentence, the test, and the decision. The hallway request will be remembered differently by the two people who heard it. Write it down, even if you then build it immediately.

Problem 2. Which three artifacts should you be able to show before CSCE 606?

Answer. A requirement a test can fail, the test, and a review or a decision log for one change. The graduate course will not spend its first weeks inventing those.